

COMP663 — AI Optimisation Techniques

Assignment 2: Classical Optimisation

Semester 2, 2026 | Weight: 20% | Individual Assessment

Item	Detail
Due Date	5:00 p.m., Friday, 21 August 2026
Submission Platform	Akoraka Learn
Assessment Type	Individual
GenAI Use	Permitted (see Section: GenAI Policy)

Late Submission Policy

Time Late	Penalty
Less than 6 hours late	5% late penalty
Between 6 and 24 hours late (up to 1 day)	12.5% late penalty
Between 24 and 48 hours late (1–2 days)	25% late penalty
Over 48 hours late (2 days or longer)	100% late penalty

Penalties are applied to the mark achieved, not the maximum available. Extensions more than 3 days require formal aegrotat application with supporting documentation before the due date.

Academic Integrity

This is an individual assessment. Each student must produce their own analysis and code. While general concepts may be discussed verbally with peers, all submitted work—including code, visualisations, optimisation results, model files, and written interpretations—must be entirely your own. Submissions containing duplicated or substantially identical content will result in zero marks for all parties involved.

You must not share your code, notebook, model files, search results, or written material with other students. Allowing another student to copy your work is also a breach of [academic integrity](#).

GenAI Policy

Generative AI (GenAI) tools are permitted in this course. You may use them for brainstorming, code debugging, writing assistance, and conceptual clarification. However, you remain fully responsible for the correctness, reproducibility, and originality of all submitted work.

You must create a GenAI Acknowledgement txt file stating which tools you used and for what purpose. If no GenAI tools were used, state this explicitly. Failure to declare GenAI use where it has been used may be treated as a breach of academic integrity.

Overview

In this assignment, you will improve the given baseline neural network model through systematic hyperparameter optimisation. You will apply one classical search method — grid search or random search — and two advanced optimisation techniques: Bayesian optimisation and neural architecture search (NAS). You need to identify and analyse important hyperparameters for each search/optimisation technique. Choose a final model to submit in the end.

The purpose is not simply to obtain the highest performance metric. You are expected to design optimisation workflows, justify the search space and computational cost, compare predictive performance, and analyse what the optimisation results reveal about the chosen model.

All code, visualisations, results, and written analysis must appear in the submitted Jupyter notebook. A separate written report is **NOT** required.

Learning Objectives

Upon completing this assignment, you should be able to:

- Identify important hyperparameters and design justified search spaces and computational budgets (e.g., number of trials, training time) for different hyperparameter optimisation techniques.
- Apply either grid search or random search to a neural network model.
- Apply Bayesian optimisation to a neural network model.
- Apply NAS to a neural network model.
- Conduct a relatively fair comparison of baseline and optimised models.
- Analyse hyperparameter outcomes, predictive performance, computational cost, and model limitations.
- Select, save, and document a final model for the hidden-test evaluation.

Dataset

Use the same released dataset as in Assignment 1. The data must not be replaced or externally augmented for this assessment.

Property	Value
Filename	forest_cover_data.csv
Observations in released training dataset	571,012
Number of input features	14
Target variable	Cover_Type
Number of classes	5 (integer values 1 to 5)

The 571,012 observations are the complete dataset released to students for model development. The hidden test set is separate and is not included in this count. The hidden test set follows the same feature schema and uses the same 5-class Cover-Type mapping. It represents the same underlying data distributions as the released dataset, but the size is different.

You may carry forward relevant preprocessing, feature engineering, metrics, or modelling decisions from Assignment 1, but you are not required to reproduce the Assignment 1 notebook or its complete experimental process.

Baseline Neural Network Model

The baseline model is a fully connected feedforward neural network (multilayer perceptron) consisting of an input layer matching the number of input features, two hidden layers containing 24 and 12 neurons respectively, and an output layer containing five neurons corresponding to the five forest cover classes. Each hidden layer uses the Sigmoid activation function, while the output layer produces five raw output values (logits). During training, these logits can be passed directly to the cross-entropy loss function, which internally applies the softmax operation. The architecture of this baseline model is given in baselineNN.ipynb. All optimisation tasks must begin from this baseline architecture. Changes to the architecture are permitted only within the declared search space in Tasks 2, 3 & 4.

Tasks

This assignment consists of six tasks.

Task 1: Baseline Model Training and Hyperparameter Identification (10 marks)

Build on the EDA completed in Assignment 1,

You should:

- Describe the preprocessing pipeline and any feature engineering you want to use in this assignment. Connect the baseline model with your pipeline.
- Define a primary evaluation metric and at least one secondary metric, explaining why they are suitable.
- Describe the baseline model training and evaluation procedure, and train the model. You can choose your own training optimisation hyperparameters (e.g., learning rate, batch size) in this task. Cross-validation is permitted but is not compulsory. **DO NOT** change the baseline neural network architecture in this task.
- Establish and report the unoptimised baseline performance using your own held-out test data.
- Identify all candidate hyperparameters (include training optimisation hyperparameters) that could be optimised in this assignment for different techniques in task 2, 3, and 4. Explain how each may affect model behaviour, performance, or computational cost.
- Define the hyperparameter types and search ranges without assuming that the best values are already known.

Task 2: Classical Hyperparameter Search (15 marks)

Choose and apply exactly one classical search method—grid search or random search—to the baseline model identified in Task 1.

You should:

- State which search method you selected and justify why it is appropriate for the declared search space and computational cost.
- Identify the hyperparameters to be optimised for your chosen search method. This should be a subset of the hyperparameters and ranges defined in Task 1.
- Declare and justify a feasible computational cost (e.g., maximum trials, or runtime) and justify your choices.
- Apply the method to the hyperparameters.
- Use the evaluation procedure and primary metric declared in Task 1 consistently throughout the search.
- Report the number of evaluated configurations or trials, the best hyperparameters, the best evaluation score, and total search runtime.
- Present the search results using a concise table.

Task 3: Bayesian Optimisation (20 marks)

Use Bayesian optimisation to tune the given neural network baseline.

- Identify the hyperparameters to be optimised in Bayesian optimisation. This should be a subset of the hyperparameters and ranges defined in Task 1.
- Define the optimisation objective and Bayesian search space. Explain how your surrogate model and acquisition process guide the optimisation.
- Use the same primary metric and evaluation procedure as in Tasks 1 and 2.
- Declare and justify a feasible computational cost (e.g., maximum trials, or runtime) and justify your choices.
- Apply the method to the hyperparameters.
- Report the best configuration, the best evaluation performance, and the total search runtime.
- Present the search results using a concise table.

Task 4: NAS (25 marks)

Use NAS to tune the given neural network baseline.

- Identify the hyperparameters to be optimised in NAS. This should be a subset of the hyperparameters and ranges defined in Task 1.
- Define the optimisation objective and NAS search space. Explain what your search strategy is and what kind of NAS technique you use.
- Your NAS search space must include architectural design choices (e.g., number of hidden layers, hidden-layer width, activation functions, different layer types like CNN or RNN, or skip connections where appropriate). Architecture changes are permitted only within the declared search space.
- Use the same primary metric and evaluation procedure as in Tasks 1, 2, and 3.
- Declare and justify a feasible computational cost. Large-scale architecture search is not expected and receives no additional credit.
- Apply the method to the hyperparameters and neural architecture.
- Report the best configuration include both hyperparameters and architecture, the best evaluation performance, and the total search runtime.
- Present the search results using a concise table.

Task 5: Performance Comparison and Analysis (20 marks)

Design and justify a comparison of the results from four tasks: the unoptimised Task 1 baseline, the best Task 2 configuration, the best Task 3 configuration, and the best Task 4 configuration. The appropriateness and fairness of your chosen comparison procedure will be assessed.

Your comparison should include:

- Compare and discuss the performance differences among the four tasks with respect to the primary metric and at least one secondary metric.
- Total search time, number of evaluated configurations or trials, and relevant model-complexity information.
- A concise summary table and at least one visualisation comparing the results from the four tasks.
- Analyse how important hyperparameters or architectural choices relate to the observed performance and computational cost.
- Discuss factors affecting fairness of comparison, including differences in search space, computational budget, architecture, and evaluation procedure.
- Select and justify the final model based on the comparison, and report its complete configuration.
- Provide class-level evaluation, discuss the final model's limitations, and save the final model in a .pth or .pt format.

Task 6: Hidden Test (10 marks)

Provide clear instructions for running your submitted final model on a hidden test CSV file and generating forest cover type predictions. The teaching team will evaluate the model on a hidden test set. The hidden test set will not be released.

Marks will be awarded according to the model's ranking among valid submissions:

- Top 10%: 10 marks.
- Below the top 10% but within the top 20%: 9 marks.
- Below the top 20% but within the top 30%: 8 marks.
- Below the top 30% but within the top 40%: 7 marks.
- Below the top 40% but within the top 50%: 6 marks.
- Remaining submissions: 5 marks, provided the model achieves a minimum performance threshold determined by the examiner.

Models that do not meet the minimum performance threshold will receive fewer than 5 marks, with marks awarded at the examiner's discretion based on performance. If the submitted model cannot be loaded or executed according to the supplied instructions, it will receive minimum marks for this task.

Submission Requirements

Create a GitHub repository to store all your submission documents. Submit a text file named `GitHub_URL.txt` containing only the repository URL. Name your repository `StudentID-COMP663A2`, replacing `StudentID` with your student ID (e.g., `12345678-COMP663A2`). The GitHub repository should contain the following documents:

File	Description
<code>StudentID_Assignment2.ipynb</code>	A fully executed Jupyter Notebook containing all code, outputs, visualisations, results, and written analysis. A separate report is not required.
Saved final ML model	The exact final model to be used for the hidden test, saved in a <code>.pth</code> or <code>.pt</code> format.
<code>preprocessing.py</code> (optional)	This submission is optional. You only need to submit it if your model depends on your customised preprocessing steps. This program will be used to preprocess the unseen test data before running the submitted classification model.
<code>run.txt</code>	Instructions for installing required packages, loading the model, processing a new CSV file with the same input features, and generating <code>Cover_Type</code> predictions. Also describe how to use your <code>preprocessing.py</code> file if you submit it. You should list packages and versions used for Bayesian optimisation, neural-network training, NAS, or model loading.
<code>GenAI_Acknowledgement.txt</code>	A short statement of which GenAI tools were used and for what purpose, or a statement that no GenAI tools were used.

Ensure that the notebook and model use relative file paths and can be run in a clean Python environment. Remove unnecessary debugging output, but retain the outputs required to assess your results. You **must not** push additional commits or modify the repository after the assignment deadline. Otherwise, a late penalty will apply.

Marking Criteria

The following table outlines the marks allocated to each task and provides general descriptors for three performance bands.

Task	Marks	High (80–100%)	Adequate (50–79%)	Low (0–49%)
Task 1: Baseline Model Training and Hyperparameter Identification	10	Preprocessing, metrics, and evaluation design are clearly described; hyperparameters and search space are clearly described and analysed; baseline performance is clearly reported.	Preprocessing, metrics, and evaluation design are partially described; hyperparameters and search space are partially described and analysed; baseline performance is partially reported.	Preprocessing, metrics, and evaluation design are missing or incorrect; hyperparameters and search space are missing or incorrect; baseline performance is missing.
Task 2: Classical Hyperparameter Search	15	One selected classical search is correctly implemented; search space, budget, results, runtime, and baseline comparison are clearly reported and interpreted.	The method produces usable results, but justification, reporting, visualisation, or comparison has minor gaps.	Search is missing or incorrect; results are incomplete, or unsupported.
Task 3: Bayesian Optimisation	20	Bayesian optimisation is correctly implemented and explained, budget well justified, and results are clearly described.	Bayesian optimisation works and produces usable results, partially explained; budget control or results description is partial.	Bayesian optimisation is missing, not explained, excessively scoped, incorrectly implemented, or not meaningfully evaluated.
Task 4: NAS	25	NAS is correctly implemented and explained, budget well justified, and results are clearly described.	NAS works and produces usable results, partially explained; budget control or results description is partial.	NAS is missing, not explained, excessively scoped, incorrectly implemented, or not meaningfully evaluated.
Task 5: Performance Comparison and Analysis	20	The results from the first 4 tasks are compared using a fair and justified design. Predictive performance, runtime, model complexity, and key hyperparameter or architectural effects are analysed with clear evidence and supported by an effective summary table and visualisation. The final-model selection is well justified and supported by class-level evaluation and a discussion of limitations.	The results from the first 4 tasks are compared and a final model is selected, but the fairness justification, performance or computational analysis, summary presentation, hyperparameter or architectural interpretation, class-level evaluation, and discussion of limitations are partially presented.	The results comparison is missing, unfair, misleading, or unsupported; required metrics, computational evidence, visual summaries, or hyperparameter or architectural analysis are substantially incomplete. Final-model selection is unclear, class-level evaluation and discussion of limitations are missing.
Task 6: Hidden Test	10	Top 30% performance on the hidden test set; model is loadable and executable.	Ranks below the top 30% but meets the minimum performance threshold; model is loadable and executable.	Ranks below the top 50%; Fails to pass the minimum performance threshold, or the submitted model is not loadable or executable.
Total	100			

Additional Guidance

- You are **not expected to exhaustively search** every possible hyperparameter combination. Marks are awarded for well-designed optimisation workflows, justified decisions, and thoughtful analysis rather than achieving the highest predictive performance.
- A larger search or more complex neural architecture does not automatically receive more credit. Search budgets must be feasible, declared, and justified.
- Distinguish changes in model family or architecture from hyperparameter effects, report uninformative trials honestly, and explain the objective, search space, budget, and results of any optimisation library in your own words.